

Ouvidoria Inteligente

1. Contexto e Objetivo

A Ouvidoria Geral do município recebe milhares de manifestações mensais em texto livre, hoje triadas apenas por busca de palavras-chave — o que falha em identificar duplicatas escritas com vocabulário diferente e em agrupar manifestações sobre o mesmo tema geral (ex.: saúde). Este trabalho constrói um protótipo de triagem semântica que resolve esses problemas usando representações vetoriais modernas: BoW, TF-IDF e Embeddings densos, além de técnicas de chunking para tratar denúncias longas.

2. Dataset

O enunciado previa um arquivo manifestacoes.json fornecido pelo professor, que não chegou a ser disponibilizado para a turma. Diante disso, construímos um dataset sintético com 40 manifestações anônimas, distribuídas entre as 5 categorias oficiais (infraestrutura, saúde, segurança, educação e meio ambiente), incluindo exatamente os textos citados no enunciado (M003, M008, M017, M022, M031), 3 pares de duplicatas semânticas propositais (~15% do corpus) e 5 manifestações longas (acima de 500 caracteres) para a etapa de chunking.

3. Entrega 1 — Análise Comparativa de Representações

Geramos representações Bag-of-Words, TF-IDF (scikit-learn) e Embeddings densos (sentence-transformers, modelo multilíngue paraphrase-multilingual-MiniLM-L12-v2) para o corpus completo. Calculamos a similaridade de cosseno para os pares indicados: M003×M017 e M008×M022 (duplicatas semânticas esperadas, mas com vocabulário quase totalmente diferente) e M008×M031 (par de controle, sobre temas distintos). A hipótese confirmada na análise é que BoW e TF-IDF tendem a subestimar a similaridade real de paráfrases — por dependerem de sobreposição literal de palavras — enquanto os Embeddings capturam a equivalência semântica mesmo sem palavras em comum, ao custo de menor interpretabilidade e maior custo computacional.

4. Entrega 2 — Detecção de Duplicatas

Implementamos a função `detectar_duplicatas(textos, limiar=0.85)`, que calcula a matriz de similaridade de cosseno completa sobre os embeddings do corpus e retorna todos os pares acima do limiar. Escolhemos 0.85 como ponto de partida — faixa em que paráfrases do mesmo evento tendem a ficar, sem capturar textos apenas "relacionados" pelo tema. Também implementamos um limiar dinâmico alternativo, baseado no percentil 90 da distribuição de similaridades do corpus, mais robusto a mudanças no tamanho ou na densidade temática da base. A avaliação comparou os pares detectados com os 3 pares de duplicatas reais conhecidas do dataset (M003/M017, M008/M022, M007/M012), calculando verdadeiros positivos, falsos positivos e falsos negativos para validar a calibração do limiar.

5. Entrega 3 — Chunking de Manifestações Longas

As 5 manifestações mais longas do corpus foram divididas com `RecursiveCharacterTextSplitter` (LangChain) em duas configurações: (`chunk_size=300`, `overlap=30`) e (`chunk_size=600`, `overlap=100`). Overlaps maiores preservam melhor a coesão entre chunks consecutivos, evitando que uma frase seja partida entre causa e consequência do relato — importante em denúncias argumentativas, onde o problema e seu impacto costumam vir em frases seguidas. A configuração de chunks maiores (600/100) se mostrou mais adequada para preservar o sentido das denúncias deste corpus, enquanto a configuração menor (300/30) é preferível quando se busca granularidade fina para apontar a frase exata de uma consulta. A visualização em PCA/t-SNE dos embeddings dos chunks confirma que chunks de uma mesma manifestação tendem a formar agrupamentos visuais próprios, próximos entre si e distantes dos chunks de manifestações sobre outros temas.

6. Entrega 4 — App Streamlit (`app_ouvidoria.py`)

A aplicação unifica as três primeiras entregas em uma interface com 4 abas: Busca Semântica (consulta livre do cidadão, retorno top-k configurável com destaque por cor conforme o score), Base Completa (tabela de todas as manifestações e geração sob demanda da matriz de similaridade/heatmap), Espaço Vetorial (PCA ou t-SNE selecionável, colorido por categoria oficial, com campo de texto para o aluno registrar se os clusters semânticos coincidem com as categorias) e Chunking (área para colar ou selecionar uma manifestação longa, escolher estratégia e parâmetros, e visualizar os chunks e seus embeddings). O modelo de embedding e o top-k são configuráveis na sidebar, com cache via `st.cache_resource` (modelo) e `st.cache_data` (embeddings), conforme recomendado no enunciado.

7. Dificuldades e Aprendizados

A principal dificuldade foi a ausência do dataset original — contornada com a construção cuidadosa de um corpus sintético que respeitasse as restrições do enunciado (categorias, duplicatas propositalmente, textos longos, pares citados nominalmente). O trabalho reforçou, na prática, uma lição central de Recuperação de Informação e PLN: representações lexicais (BoW/TF-IDF) são rápidas e interpretáveis, mas cegas a sinônimos e paráfrases, enquanto embeddings resolvem esse problema à custa de maior custo computacional, dependência do modelo pré-treinado escolhido e menor interpretabilidade individual dos scores.

8. Divisão de Tarefas

Integrante	Responsabilidade
Bruno Cavalcante Moura Vieira Filho	Construção do dataset sintético (<code>manifestacoes.json</code>), Entrega 1 (análise comparativa BoW/TF-IDF/Embeddings) e Entrega 2 (função <code>detectar_duplicatas</code> , limiar e análise de falsos positivos/negativos)
Miguel Giovanni Porto Vieira	Entrega 3 (chunking das manifestações longas e comparação de configurações), Entrega 4 (app Streamlit <code>app_ouvidoria.py</code> com as 4 abas) e elaboração deste relatório